

第 1 章 线性规划和整数规划

1.1 线性规划

定义 1.1

为了指定最优的生产计划，最大经济效益、最小运输成本等目标，线性规划通过构建线性目标函数和线性约束条件来求解最优解。



题 1.1

工厂中具有这样的资源条件：

资源	产品			可用数量
	P_1	P_2	P_3	
原材料 A	2	4	3	150
原材料 B	3	1	5	160
加工时间 C	7	3	5	200
产品利润	70	50	60	

解 可以建立该问题的数学模型为

$$\begin{aligned} \max z &= 70x_1 + 50x_2 + 60x_3, \\ \text{s.t.} \quad &\begin{cases} 2x_1 + 4x_2 + 3x_3 \leq 150, \\ 3x_1 + x_2 + 5x_3 \leq 160, \\ 7x_1 + 3x_2 + 5x_3 \leq 200, \\ x_i \geq 0, i = 1, 2, 3. \end{cases} \end{aligned}$$

这里的 s. t. (subject to 的缩写) 是 "受约束于" 的意思。

此处，规划问题中的方案数、措施等是 **决策变量**，而目标函数和约束条件则是 **线性函数**。

定理 1.1

一般线性规划的模型如下：

$$\text{s.t.} \quad \begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \leq (\text{或 } =, \geq) b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \leq (\text{或 } =, \geq) b_2, \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \leq (\text{或 } =, \geq) b_m, \\ x_1, x_2, \cdots, x_n \geq 0. \end{cases}$$

或简写为

$$\begin{aligned} \max(\text{或 } \min) z &= \sum_{j=1}^n c_j x_j, \\ \text{s.t.} \quad &\begin{cases} \sum_{j=1}^n a_{ij} x_j \leq (\text{或 } =, \geq) b_i, & i = 1, 2, \cdots, m, \\ x_j \geq 0, & j = 1, 2, \cdots, n. \end{cases} \end{aligned}$$



1.1.1 连续线性规划

题 1.2 一个奶制品加工厂用牛奶生产 A_1 和 A_2 两种奶制品，一桶牛奶可以在甲车间可以用 $12h$ 加工成 $3kg A_1$ ，或者在乙车间用 $8h$ 加工成 $4kg A_2$ 。根据市场需求，生产出来的 A_1 和 A_2 能够全部售出，且售出每千克 A_1 获利 24 元，售出每千克 A_2 获利 16 元。现在工厂每天能得到 50 桶牛奶，每天正式工人总共劳动时间 $480h$ ，并且甲车间的设备每天之多能加工 $100kg A_1$ ，乙车间的的加工能力没有上线（加工）能力足够大，试为该厂定制一个生产计划，使得每天获利最大。

解

$$\begin{aligned} \max f &= 72x_1 + 64x_2 \\ \text{s.t.} \quad &\begin{cases} 1 + x_2 \leq 50 \\ 12x_1 + 8x_2 \leq 480 \\ 3x_1 \leq 100 \\ x_1 \geq 0, x_2 \geq 0 \end{cases} \end{aligned}$$

这是因为牛奶是可分的，所以看成是连续线性规划。

💡 Tips

这里采用当前最热门的线性规划库来进行求解：

```

1 def lp() -> tuple:
2     """
3     求解线性规划问题，最大化目标函数
4     :return: tuple, 包含状态、x1、x2的最优解和目标函数的最值。
5     """
6     # 决策变量定义，注意定义变量的类型
7     x1 = pulp.LpVariable('x1', lowBound=0, cat='Continuous')
8     x2 = pulp.LpVariable('x2', lowBound=0, cat='Continuous') # 限制约束变量为连续性
9
10    model = pulp.LpProblem('Question-1-Maximize', pulp.LpMaximize)
11    # model = pulp.LpProblem("Question-2-Minimize", pulp.LpMinimize) # 如果要求最小值，就用 LpMinimize 参数
12    model += 72 * x1 + 64 * x2
13
14    # 约束条件
15    model += x1 + x2 <= 50
16    model += 12 * x1 + 8 * x2 <= 480
17    model += 3 * x1 <= 100
18
19    model.solve()
20
21    return pulp.LpStatus[model.status], x1.varValue, x2.varValue, pulp.value(model.objective)

```

Listing 1.1: 线性规划的 Python 代码

1.1.2 整数线性规划

1.1.3 0-1 规划

定义 1.2

如果变量只有可取和不可取两种选择，那么就是 0-1 规划



此时可以使用代码：

```

1 x1 = pulp.LpVariable('x1', cat='Binary') # Binary 表示0-1变量
2 x2 = pulp.LpVariable('x2', cat='Binary')

```

Listing 1.2: 0-1 规划的 Python 代码

题 1.3 轮班问题

假设每个时间段都需要一定的工人，而每个工人只能工作 8 小时。

班次	1	2	3	4	5	6
时间段	0:00-4:00	4:00-8:00	8:00-12:00	12:00-16:00	16:00-20:00	20:00-24:00
工人数量	35	40	50	45	55	30

模型假设：

1. 每名工人每 24 h 只能工作 8 h。
2. 每名工人只能在某个班次的初始时刻报到。

```

1 def lp() -> tuple:
2     x = pulp.LpVariable.dicts("x", range(1, 7), lowBound=0, cat='Integer')
3     # 注意, lowBound = 0 已经定义了 x_i >= 0 的约束条件, 所以后续模型无需再单独添加这些约束条件。
4
5     model = pulp.LpProblem('轮班问题', pulp.LpMinimize)
6     model += pulp.lpSum(x.values()) # 目标函数: 最小化所有班次的总和
7
8     # 约束条件
9     model += x[1] + x[6] >= 35
10    model += x[1] + x[2] >= 40
11    model += x[2] + x[3] >= 50
12    model += x[3] + x[4] >= 45
13    model += x[4] + x[5] >= 55
14    model += x[5] + x[6] >= 30
15
16    model.solve()
17    values = {f"x{i}": x[i].varValue for i in range(1, 7)}
18    return pulp.LpStatus[model.status], values, pulp.value(model.objective)

```

Listing 1.3: 0-1 规划的 Python 代码

1.1.3.1 二维 0-1 规划

题 1.4 指派问题 某单位有 n 项任务, 正好需 n 个人去完成, 由于每项任务的性质和每个人的能力和专长的不同, 假设分配每个人只能完成一项任务。设 c_{ij} 表示分配第 i 个人去完成第 j 项任务的费用 (时间等), 问应如何指派, 完成任务的总费用最小?

解 引进 0-1 变量

$$x_{ij} = \begin{cases} 1, & \text{指派第 } i \text{ 个人完成第 } j \text{ 项任务,} \\ 0, & \text{不指派第 } i \text{ 个人完成第 } j \text{ 项任务,} \end{cases} \quad i, j = 1, 2, \dots, n.$$

目标函数: 总费用最小

$$\min z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$$

约束条件:

1. 每个人只能安排 1 项任务: $\sum_{j=1}^n x_{ij} = 1, i = 1, 2, \dots, n$;
2. 每项任务只能指派 1 个人完成: $\sum_{i=1}^n x_{ij} = 1, j = 1, 2, \dots, n$;
3. 0-1 条件: $x_{ij} = 0$ 或 $1, i, j = 1, 2, \dots, n$ 。

这里的约束条件给的很精妙, 因为对 0-1 变量求和为定值 1, 直接描述出了每个人只能安排一个任务。

 Tips

题 1.5 旅行商问题 有一推销员, 从城市 v_1 出发, 要遍访城市 $v_2, v_3, \dots, v_n$ 各一次, 最后返回 v_1 。已知从 v_i 到 v_j 的旅费为 c_{ij} , 试问应按怎样的次序访问这些城市, 使得总旅费最少?

解 模仿上道题的解法, 我们定义:

决策变量: 对每一对城市 v_i, v_j , 定义一个变量 x_{ij} 来表示是否要从 v_i 出发访问 v_j , 令:

$$x_{ij} = \begin{cases} 1, & \text{推销员决定从 } v_i \text{ 直接进入 } v_j, \\ 0, & \text{推销员不从 } v_i \text{ 直接进入 } v_j, \end{cases}$$

式中: $i, j = 1, 2, \dots, n$ 。目标函数: 若推销员决定从 v_i 直接进入 v_j , 则由已知, 其旅行费用为 $c_{ij}x_{ij}$, 于是总旅费可表达为:

$$z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij},$$

其中, 若 $i = j$, 则规定 $c_{ii} = M, M$ 为事先选定的充分大实数, $i, j = 1, 2, \dots, n$ 。约束:

(1) 每个城市恰好进入一次：

$$\sum_{i=1}^n x_{ij} = 1, \quad j = 1, 2, \dots, n.$$

(2) 每个城市离开一次：

$$\sum_{j=1}^n x_{ij} = 1, \quad i = 1, 2, \dots, n.$$

(3) 为防止在遍历过程中，出现子回路，附加一个强制性约束：

$$u_i - u_j + nx_{ij} \leq n - 1, \quad i = 1, \dots, n, j = 2, \dots, n,$$

式中： $u_1 = 0, 1 \leq u_i \leq n - 1, i = 2, 3, \dots, n$ 。综上所述，建立旅行商问题的如下 0-1 整数规划模型：

$$\begin{aligned} \min z &= \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}, \\ \text{s.t.} \quad &\begin{cases} \sum_{i=1}^n x_{ij} = 1, & j = 1, 2, \dots, n, \\ \sum_{j=1}^n x_{ij} = 1, & i = 1, 2, \dots, n, \\ u_i - u_j + nx_{ij} \leq n - 1, & i = 1, \dots, n, j = 2, \dots, n, \\ u_1 = 0, 1 \leq u_i \leq n - 1, & i = 2, 3, \dots, n, \\ x_{ij} = 0 \text{ 或 } 1, & i, j = 1, 2, \dots, n. \end{cases} \end{aligned}$$

这里最难点地方在于，第三个不等式应该如何理解？

💡 Tips

实际上，关键是任何不包含起点的子回路都不可能满足 MTZ 约束。

唯一满足所有约束的解，必然包含所有城市、从起点出发并最终返回起点的单一完整回路。

解 情况一：路线中不包含 $i \rightarrow j$ 的路径 ($x_{ij} = 0$)

- 不等式变为： $u_i - u_j + n \cdot 0 \leq n - 1 \implies u_i - u_j \leq n - 1$
- 由于我们已经约束了 $0 \leq u_i \leq n - 1$ 和 $0 \leq u_j \leq n - 1$ ，那么 $u_i - u_j$ 的最大可能值就是 $(n - 1) - 0 = n - 1$
- 所以， $u_i - u_j \leq n - 1$ 这个条件永远成立
- 结论：如果旅行商不从城市 i 走到城市 j ，那么这个约束是“无效”的，不会对解产生任何限制

情况二：路线中包含了 $i \rightarrow j$ 的路径 ($x_{ij} = 1$)

- 不等式变为： $u_i - u_j + n \cdot 1 \leq n - 1 \implies u_i - u_j \leq -1 \implies u_i + 1 \leq u_j$
- 这就是关键所在！这个结果强制要求：如果旅行商从城市 i 直接走到了城市 j ，那么城市 j 的“位次” u_j 必须至少比城市 i 的“位次” u_i 大 1
- 结论：这个约束巧妙地将路径选择 (x_{ij}) 和访问顺序 (u_i) 关联了起来，强迫 u_i 沿着旅行路线单调递增

1.2 非线性规划

线性规划的约束可以视为有多种约束形式，有线性约束、等式约束和非线性约束。

💡 Tips

非线性规划无法使用 `linprog` 函数求解，需要使用科学计算库：`scipy.optimize`，使用 `minimize` 函数求解。

题 1.6 默认求解的是 $f(x)$ 的最小值，若需要求解最大值，可以将目标函数取负数。

$$\begin{cases} f(x)_{\min} = x_1^2 + x_2^2 + x_3^2 + 8 \\ x_1^2 - x_2 + x_3^2 \geq 0 \\ x_1 + x_2^2 + x_3^3 \leq 0 \\ -x_1 - x_2^2 + 2 = 0 \\ x_2 + 2x_3^2 = 3 \\ x_1, x_2, x_3 \geq 0 \end{cases}$$

解

1.2.1 非线性规划的数学模型

非线性规划还可能在影院观影角度、飞行管理等方面应用。

定义 1.3 (非线性规划的数学模型)

记 $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ 是 n 维欧几里得空间 R^n 中的一个点 (n 维向量)。 $f(\mathbf{x})$, $g_i(\mathbf{x}), i = 1, 2, \dots, p$ 和 $h_j(\mathbf{x}), j = 1, 2, \dots, q$ 是定义在 R^n 上的实值函数。

若 $f(\mathbf{x}), g_i(\mathbf{x}), i = 1, 2, \dots, p$ 和 $h_j(\mathbf{x}), j = 1, 2, \dots, q$ 中至少有一个是 $\mathbf{x}$ 的非线性函数, 称如下形式的数学模型:

$$\begin{aligned} \min & f(\mathbf{x}) \\ \text{s.t.} & \begin{cases} g_i(\mathbf{x}) \leq 0, & i = 1, 2, \dots, p, \\ h_j(\mathbf{x}) = 0, & j = 1, 2, \dots, q. \end{cases} \end{aligned}$$

为非线性规划模型的一般形式。如果采用向量表示法, 则非线性规划的一般形式还可以写为

$$\begin{aligned} \min & f(\mathbf{x}), \\ \text{s.t.} & \begin{cases} \mathbf{G}(\mathbf{x}) \leq 0, \\ \mathbf{H}(\mathbf{x}) = 0, \end{cases} \end{aligned}$$

$$\text{式中: } \mathbf{G}(\mathbf{x}) = [g_1(\mathbf{x}), g_2(\mathbf{x}), \dots, g_p(\mathbf{x})]^T, \mathbf{H}(\mathbf{x}) = [h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_q(\mathbf{x})]^T$$



如果线性规划的最优解存在, 则最优解只能在可行域的边界上达到。且求出的是全局最优解, 但非线性规划可能在可行域内的任何一点达到, 而一般的线性规划算法给出的只是局部最优解。

💡 Tips

定义 1.4 (全局最优值)

- 若 $\mathbf{x}^* \in K$, 且 $\forall \mathbf{x} \in K$, 都有 $f(\mathbf{x}^*) \leq f(\mathbf{x})$, 则称 $\mathbf{x}^*$ 为全局最优解, 称 $f(\mathbf{x}^*)$ 为其全局最优值。如果 $\forall \mathbf{x} \in K, \mathbf{x} \neq \mathbf{x}^*$, 都有 $f(\mathbf{x}^*) < f(\mathbf{x})$, 则称 $\mathbf{x}^*$ 为严格全局最优解, 称 $f(\mathbf{x}^*)$ 为其严格全局最优值。
- 若 $\mathbf{x}^* \in K$, 且存在 $\mathbf{x}^*$ 的邻域 $N_\delta(\mathbf{x}^*)$, $\forall \mathbf{x} \in N_\delta(\mathbf{x}^*) \cap K$, 都有 $f(\mathbf{x}^*) \leq f(\mathbf{x})$, 则称 $\mathbf{x}^*$ 为局部最优解, 称 $f(\mathbf{x}^*)$ 为其局部最优值。如果 $\forall \mathbf{x} \in N_\delta(\mathbf{x}^*) \cap K, \mathbf{x} \neq \mathbf{x}^*$, 都有 $f(\mathbf{x}^*) < f(\mathbf{x})$, 则称 $\mathbf{x}^*$ 为严格局部最优解, 称 $f(\mathbf{x}^*)$ 为其严格局部最优值。



1.2.2 利用 Pyomo 与求解器进行求解

Pyomo 是一个编程语言, 致力于编写非线性约束优化问题, 而运行 Pyomo 需要安装一个求解器。

如果是线性规划, 直接用 pulp 库就行, 但是非线性规划目前还是需要调用求解器。

1.2.3 利用 Minimize 模块求解

Scipy 模块不能运行求解器, 也不能运行整数决策变量。

Constraint 是约束, Jac 是一阶导数信息, Hess 是二阶导数信息, 可以输入 Method、Bounds 等。

定理 1.2 (约束条件)

- 等式约束: `lambda: x[0] + x[1] - 5`
- 不等式约束: 默认采用 Non-Negative 非零约束, 如 `lambda: x[0] - 2*x[1] + 2`



题 1.7

$$\begin{aligned} \min_{\mathbf{x}} & q(\mathbf{x}) = (x_1 - 1)^2 + (x_2 - 2.5)^2 \\ \text{subject to} & \begin{aligned} & x_1 - 2x_2 + 2 \geq 0 \\ & -x_1 - 2x_2 + 6 \geq 0 \\ & -x_1 + 2x_2 + 2 \geq 0 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \end{aligned} \end{aligned}$$

```
1 def example():
2     func = lambda x: (x[0] - 1) ** 2 + (x[1] - 2.5) ** 2
3     x0 = np.asarray([0, 0]) # 设定初始解
4     bounds = np.asarray([[0, None], [0, None]]) # 无上界, x[0], x[1] >= 0
5     cons = ({'type': 'ineq', 'fun': lambda x: x[0] - 2 * x[1] + 2}, # x[0] - 2*x[1] + 2 >= 0
6             {'type': 'ineq', 'fun': lambda x: -x[0] - 2 * x[1] + 6}, # -x[0] - 2*x[1] + 6 >= 0)
```

```

7         {'type': 'ineq', 'fun': lambda x: -x[0] + 2 * x[1] + 2}) # -x[0] + 2*x[1] + 2 >= 0
8
9     res = minimize(func, x0, bounds=bounds, constraints=cons)
10    return res

```

1.2.4 解决 MIP 问题

定义 1.5 (MIP 问题)

混合整数规划问题 (Mixed Integer Programming, MIP) 是指决策变量中既有连续变量又有离散变量 (通常是整数变量或二进制变量) 的数学规划问题。一般形式为:

$$\begin{aligned}
 & \min_{\mathbf{x}, \mathbf{y}} f(\mathbf{x}, \mathbf{y}) \\
 & \text{s.t. } g_i(\mathbf{x}, \mathbf{y}) \leq 0, \quad i = 1, 2, \dots, m \\
 & \quad h_j(\mathbf{x}, \mathbf{y}) = 0, \quad j = 1, 2, \dots, p \\
 & \quad \mathbf{x} \in \mathbb{R}^n \\
 & \quad \mathbf{y} \in \mathbb{Z}^q
 \end{aligned}$$

其中 $\mathbf{x}$ 是连续变量向量, $\mathbf{y}$ 是整数变量向量。当目标函数和约束条件均为线性时, 称为混合整数线性规划 (MILP); 当存在非线性函数时, 称为混合整数非线性规划 (MINLP)。



整数问题和线性问题的解法有很大的不一致, 我们常猜测整数问题是最接近线性解决方案可行域中的整数点, 但事实并非如此。

```

1 from ortools.linear_solver import pywraplp
2 def main():
3     solver = pywraplp.Solver.CreateSolver("SAT")
4     if not solver:
5         return
6
7     infinity = solver.infinity()
8     x = solver.IntVar(0.0, infinity, "x")
9     y = solver.IntVar(0.0, infinity, "y")
10    solver.Add(x + 7 * y <= 17.5)
11    solver.Add(x <= 3.5)
12    solver.Maximize(x + 10 * y)
13    status = solver.Solve()
14
15    if status == pywraplp.Solver.OPTIMAL:
16        print("Objective value =", solver.Objective().Value())
17        print("x =", x.solution_value())
18        print("y =", y.solution_value())
19    else:
20        print("The problem does not have an optimal solution.")

```